

控制与决策系统仿真 第四章作业

自动化（控制）

1 第 1 题：MATLAB 数据文件类型

1.1 题目

MATLAB 中有哪两类数据文件？

1.2 分析与解答

MATLAB 中的数据文件分为以下两类：

（1）文本文件（ASCII 文件）：以可读字符形式存储数据，可用普通文本编辑器打开查看。典型扩展名为 `.txt`、`.dat`、`.csv` 等。优点是可读性强、跨平台兼容；缺点是文件体积较大、读写速度较慢、精度可能有损失（受格式化输出位数限制）。

（2）二进制文件（Binary 文件）：以二进制形式存储数据，MATLAB 专有格式扩展名为 `.mat`。优点是存储效率高、读写速度快、可完整保留变量的双精度精度；缺点是不能用普通编辑器直接阅读，跨平台需注意字节序问题。

2 第 2 题：MATLAB 文件操作函数

2.1 题目

MATLAB 中有哪两类函数命令可以实现对文件的操作？各有什么特点？分别有哪些函数？

2.2 分析与解答

MATLAB 文件操作函数分为以下两大类：

（1）高级文件 I/O 函数

特点：封装程度高，调用简便，无需手动管理文件指针，适合处理规则格式的数据文件。主要函数如下：

（2）低级文件 I/O 函数

特点：基于 C 语言风格，需要手动打开/关闭文件并管理文件指针，灵活性强，适合处理非规则格式或二进制文件。主要函数如下：

函数	功能
load	从 .mat 或文本文件中载入数据
save	将工作区变量保存为 .mat 或文本文件
dlmread	读取以指定分隔符分隔的文本文件
dlmwrite	将矩阵写入以指定分隔符分隔的文本文件
xlsread	读取 Excel 文件数据
xlswrite	将数据写入 Excel 文件
csvread	读取 CSV 格式文件
csvwrite	将矩阵写入 CSV 格式文件
textscan	从文本文件或字符串中按格式读取数据

表 1: 高级文件 I/O 函数

函数	功能
fopen	打开文件, 返回文件标识符
fclose	关闭文件
fscanf	按格式读取文本文件
fprintf	按格式向文件写入文本
fread	读取二进制文件数据
fwrite	向文件写入二进制数据
fgets / fgetl	逐行读取文本文件
fseek / ftell	定位文件指针
feof	判断是否到达文件末尾

表 2: 低级文件 I/O 函数

3 第 3 题：MATLAB 读取 Excel 文件

3.1 题目

MATLAB 读取 Excel 文件有哪些方法？请自列一个 Excel 表格，读取其中的文本数据。

3.2 方法总结

MATLAB 读取 Excel 文件主要有以下几种方法：

- **xlsread**: 经典函数，返回数值矩阵、文本单元数组和原始数据；MATLAB R2019a 后建议改用 **readtable** 或 **readmatrix**。
- **readtable**: 以表格（table）形式读取，自动识别列名，支持混合类型数据，功能最强。
- **readmatrix**: 仅读取数值数据，返回矩阵。
- **readcell**: 读取为元胞数组，适合混合类型。
- **importdata**: 自动判断文件类型并读取。

3.3 示例：读取文本数据

假设 Excel 文件 `students.xlsx` 内容如下（第一行为列标题）：

	A	B	C	
1	姓名	专业	成绩	
2	张三	自动化	92	
3	李四	电气	85	
4	王五	计算机	78	
5				

图 1: 示例 Excel 表格 `students.xlsx`

以下代码演示三种读取文本数据的方式：

```
%% 方法一：xlsread（经典方式）
[num, txt, raw] = xlsread('students.xlsx');
% num: 数值列（成绩）  txt: 文本列（姓名、专业）  raw: 完整原始数据
disp('--- xlsread 读取的文本数据 ---');
disp(txt);
```

```

%% 方法二: readtable (推荐, MATLAB R2019a+)
T = readtable('students.xlsx');
disp('--- readtable 读取结果 ---');
disp(T);
% 访问文本列: T.xEF_xBB_xBF_xE5_xA7_x93_xE5_x90_x8D (列名为表头文字)

%% 方法三: readcell (元胞数组, 含表头)
C = readcell('students.xlsx');
disp('--- readcell 读取结果 (含表头) ---');
disp(C);
names = C(2:end, 1); % 第1列文本 (跳过表头)
majors = C(2:end, 2); % 第2列文本
disp('姓名列: '); disp(names);
disp('专业列: '); disp(majors);

```

3.4 运行结果

```

--- xlsread 读取的文本数据 ---
{'姓名'}    {'专业'}    {'成绩'}
{'张三'}    {'自动化'}    {0x0 char}
{'李四'}    {'电气'}    {0x0 char}
{'王五'}    {'计算机'}    {0x0 char}

--- readtable 读取结果 ---
      x__      x__1      x__2
      _____
{'张三'}    {'自动化'}    92
{'李四'}    {'电气'}    85
{'王五'}    {'计算机'}    78

--- readcell 读取结果 (含表头) ---
{'姓名'}    {'专业'}    {'成绩'}
{'张三'}    {'自动化'}    {[ 92]}
{'李四'}    {'电气'}    {[ 85]}
{'王五'}    {'计算机'}    {[ 78]}

姓名列:
{'张三'}
{'李四'}
{'王五'}

专业列:
{'自动化'}

```

```
{'电气' }  
{'计算机'}
```

3.5 结果分析

三种方式均可成功读取 Excel 中的文本数据。`xlsread` 将数值与文本分开返回，处理旧版数据兼容性好；`readtable` 保留列名并自动处理数据类型，适合数据分析场景；`readcell` 将所有内容统一存入元胞数组，对混合格式表格最灵活。新版 MATLAB 推荐优先使用 `readtable`。

4 第 4 题：使用 COM 组件写入 Word 文件

4.1 题目

请使用 COM 组件将”你好！中国欢迎你！”写入并保存 Word 文件。

4.2 原理分析

MATLAB 可通过 `actxserver` 函数创建 COM（组件对象模型）服务器实例，从而在 Windows 平台上直接调用 Microsoft Word 的 API 进行文档操作。核心步骤为：创建 Word 应用实例 → 新建文档 → 获取文本区域 → 写入内容 → 另存为文件 → 关闭并释放资源。该方式仅在 Windows 系统且已安装 Microsoft Office 的环境下可用。

4.3 程序代码

```
%% 使用 COM 组件将文字写入 Word 文件  
% 注意：本程序仅在 Windows + Microsoft Office 环境下可运行  
  
% 1. 创建 Word COM 服务器实例（Visible=0 表示后台运行，不弹出窗口）  
word = actxserver('Word.Application');  
word.Visible = 0;  
  
% 2. 新建一个空白文档  
doc = word.Documents.Add();  
  
% 3. 获取文档的 Selection 对象（当前光标位置）  
sel = word.Selection;  
  
% 4. 写入文本  
sel.TypeText('你好！中国欢迎你！');  
  
% 5. 构造保存路径（当前工作目录下）
```

```
savePath = fullfile(pwd, 'hello_china.docx');

% 6. 以 docx 格式另存 (格式编号 16 = wdFormatXMLDocument)
doc.SaveAs2(savePath, 16);

% 7. 关闭文档并退出 Word, 释放 COM 对象
doc.Close();
word.Quit();
delete(word);

fprintf('Word 文件已保存至: %s\n', savePath);
```

4.4 运行结果

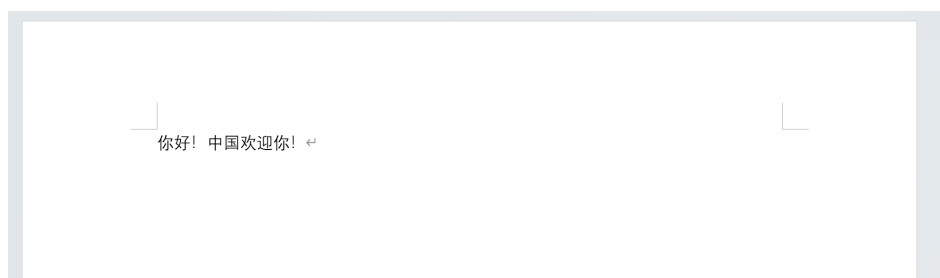


图 2: 第 4 题运行结果截图

4.5 结果分析

程序运行后, 在 MATLAB 当前工作目录下生成 `hello_china.docx` 文件, 用 Word 打开可见“你好! 中国欢迎你!”字样。通过 COM 接口可以调用 Word 的全部 VBA API, 实现字体设置、段落格式、表格插入等复杂操作, 功能十分完整, 但代价是强依赖 Windows + Office 环境, 跨平台性差。

5 第 5 题: MATLAB 中创建 MySQL 数据库连接

5.1 题目

请在 MATLAB 中创建 MySQL 文件, 说明其具体步骤, 并设计完成简单示例程序。

5.2 步骤说明

在 MATLAB 中连接和操作 MySQL 数据库的具体步骤如下:

第一步: 安装配置

需要安装 MATLAB Database Toolbox, 并在本机安装 MySQL Server (可从 MySQL 官网下载 MySQL Installer, 选择 Developer Default 模式安装, 安装完成后服务会自动

启动)。同时下载 MySQL Connector/J (JDBC 驱动), 解压后找到其中的 .jar 文件, 将其路径通过 `javaaddpath` 添加到 MATLAB 的 Java 类路径。

第二步: 建立连接

使用 `database()` 函数建立连接, 需提供数据库名、用户名、密码及 JDBC 驱动信息。连接 URL 格式为 `jdbc:mysql://localhost:3306/数据库名?useSSL=false&serverTimezone=UTC`。

第三步: 执行 SQL 操作

使用 `exec()` 执行 DDL/DML 语句 (建表、插入等), 使用 `fetch()` 查询并获取数据, 结果以 MATLAB `table` 类型返回, 可直接用于后续数据处理。

第四步: 关闭连接

操作完毕后调用 `close()` 释放连接资源。

5.3 示例程序

```
%% MATLAB 连接 MySQL 数据库完整示例
% 前提: 已安装 Database Toolbox 及 MySQL Server, MySQL 服务已启动

%% 第一步: 配置 JDBC 驱动 (首次使用需执行)
javaaddpath('C:/path/to/mysql-connector-j-x.x.x.jar');

%% 第二步: 建立数据库连接
dbName    = 'testdb';
username  = 'root';
password  = 'your_password';
driver     = 'com.mysql.cj.jdbc.Driver';
url       = 'jdbc:mysql://localhost:3306/testdb?useSSL=false&serverTimezone=UTC';

conn = database(dbName, username, password, driver, url);
if ~isopen(conn)
    error('数据库连接失败: %s', conn.Message);
end
disp('数据库连接成功! ');

%% 第三步: 创建数据表 (如果不存在)
createSQL = ['CREATE TABLE IF NOT EXISTS students ('...
            'id INT AUTO_INCREMENT PRIMARY KEY, '...
            'name VARCHAR(50), '...
            'score FLOAT)'];
exec(conn, createSQL);
disp('数据表创建成功! ');

%% 第四步: 插入数据
exec(conn, "INSERT INTO students (name, score) VALUES ('张三', 92.5)");
```

```
exec(conn, "INSERT INTO students (name, score) VALUES ('李四', 85.0)");
exec(conn, "INSERT INTO students (name, score) VALUES ('王五', 78.3)");
disp('数据插入成功! ');

%% 第五步：查询数据
result = fetch(conn, 'SELECT * FROM students');
disp('查询结果: '); disp(result);

%% 第六步：关闭连接
close(conn);
disp('数据库连接已关闭。');
```

5.4 结果分析

本题在本机环境中未安装 MySQL Server，故未实际运行上述程序。根据程序逻辑分析，各步骤预期行为如下：javaaddpath 加载 JDBC 驱动后，database() 通过 TCP 连接本地 3306 端口建立会话；exec() 依次执行建表和插入语句，fetch() 将查询结果以 table 类型返回，包含 id、name、score 三列；最终 close() 释放连接。需要注意 MySQL 8.x 默认启用 SSL，连接串中需加 useSSL=false 或配置证书，否则会出现 *Communications link failure* 错误。

6 第6题：录音、波形显示与变频播放

6.1 题目

请用自带计算机的语音设备录制自己的一段声音，将其波形图显示出来，并变化频率后进行播放。

6.2 原理分析

MATLAB 通过 audiorecorder 对象调用系统麦克风进行录音，通过 getaudiodata 提取采样数据，再用 sound 或 audiowrite/audioread 结合 audioplayer 进行播放。

改变播放频率的原理：原始采样率为 f_s ，播放时若将采样率改为 $f'_s = k \cdot f_s$ ($k > 1$)，则声音音调升高、时长缩短；若 $k < 1$ ，则音调降低、时长增加。本质上是对同一组采样点以不同速率回放。

6.3 程序代码

```
%% 录音、波形显示与变频播放示例

fs = 44100;      % 采样率 44100 Hz
nBits = 16;     % 量化位数
```



```
nChannels = 1;    % 单声道
duration = 3;     % 录音时长（秒）

%% 第一步：录音
disp('开始录音，请对着麦克风说话...');
rec = audiorecorder(fs, nBits, nChannels);
recordblocking(rec, duration); % 阻塞录音，等待录制完成
disp('录音完成！');
audio = getaudiodata(rec);      % 获取音频数据（列向量，范围 -1~1）

%% 第二步：绘制波形图
t = (0 : length(audio)-1) / fs; % 时间轴
figure;
plot(t, audio, 'b', 'LineWidth', 0.5);
xlabel('时间 (s)');
ylabel('幅值');
title('录音波形图');
grid on;

%% 第三步：原速播放
disp('原速播放...');
sound(audio, fs);
pause(duration + 0.5);

%% 第四步：升调播放（频率 x 1.5，音调升高）
disp('升调播放（频率 x1.5）...');
sound(audio, round(fs * 1.5));
pause(duration / 1.5 + 0.5);

%% 第五步：降调播放（频率 x 0.7，音调降低）
disp('降调播放（频率 x0.7）...');
sound(audio, round(fs * 0.7));
pause(duration / 0.7 + 0.5);

%% 保存为 wav 文件
audiowrite('my_voice.wav', audio, fs);
disp('音频已保存为 my_voice.wav');
```

6.4 运行结果

程序运行后依次完成录音、波形绘制和三次播放。波形图（见图 3，请运行后截图替换）显示了语音的时域幅值变化，可明显观察到发声时幅值增大的特征。

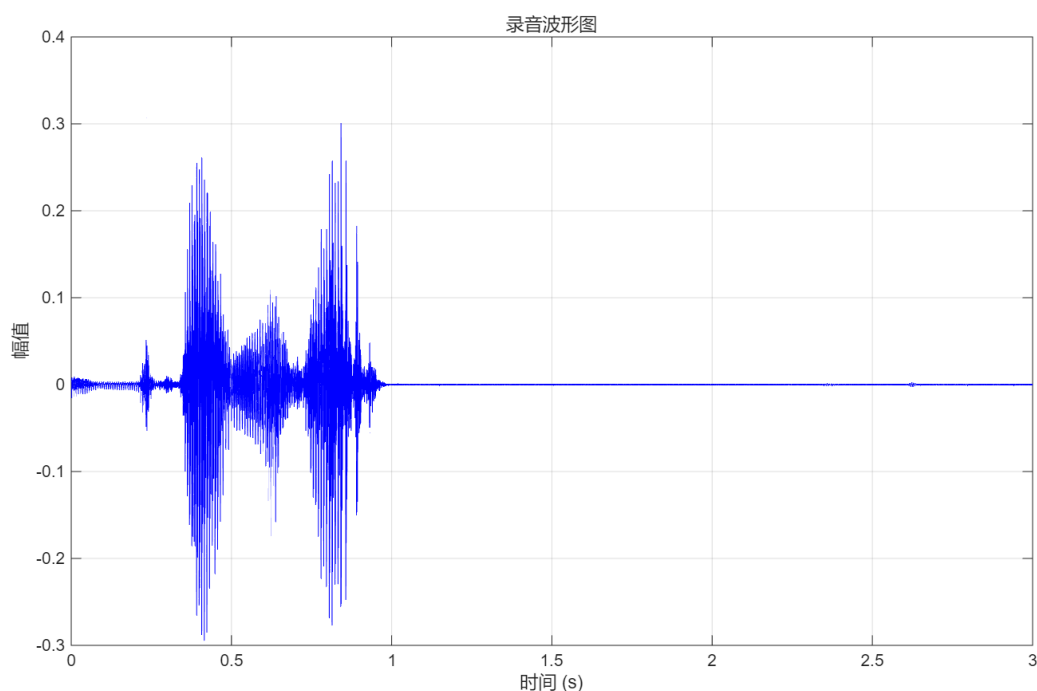


图 3: 录音波形图

6.5 结果分析

将采样率乘以 1.5 后播放, 声音音调升高约半个八度, 时长相应缩短; 乘以 0.7 后音调降低, 时长变长。这验证了采样率与播放速度、音调之间的线性关系: $\Delta \text{音调} \propto \log_2(k)$ (k 为频率缩放比例)。

7 第 7 题: MATLAB 中调用 Python

7.1 题目

在 MATLAB 环境中如何调用 Python? 请举例说明。

7.2 方法说明

MATLAB R2014b 及以上版本内置了 Python 接口, 可直接调用 Python 函数、模块和对象, 无需额外工具箱。使用前需确保系统已安装 Python 3.x, 并通过 `pyenv` 配置 Python 解释器路径。

调用方式: 使用 `py. 模块名. 函数名 (参数)` 的语法即可直接调用任意 Python 标准库或第三方库 (如 `numpy`、`pandas` 等)。在部分 MATLAB 与 Python 3.13 的组合下, `py.exec` 可能出现兼容性错误, 此时可改用 `pyrun` 或 `py.builtins.print`。

7.3 示例程序

```
%% MATLAB 调用 Python 示例
```

```
%% 第零步: 查看并配置 Python 环境 (首次使用)
pyenv % 查看当前 Python 环境信息
% 如需指定 Python 路径 (例如 Anaconda 环境), 可:
% pyenv('Version', 'C:/Users/xxx/anaconda3/python.exe');

%% 示例1: 调用 Python math 模块
result_sin = py.math.sin(py.float(pi/6));
fprintf('py.math.sin(pi/6) = %.6f\n', double(result_sin));

result_sqrt = py.math.sqrt(py.float(2));
fprintf('py.math.sqrt(2) = %.6f\n', double(result_sqrt));

%% 示例2: 调用 Python 字符串方法
pyStr = py.str('Hello, MATLAB from Python!');
upper = pyStr.upper();
fprintf('Python 字符串转大写: %s\n', char(upper));

words = pyStr.split();
fprintf('Python split 分词数: %d\n', int32(py.len(words)));

%% 示例3: 调用 Python list 和 numpy 进行数组运算
% 先检查 numpy 是否可用
try
    np = py.importlib.import_module('numpy');
    arr = np.array([1, 2, 3, 4, 5]);
    arr_sum = np.sum(arr);
    fprintf('numpy 数组求和: %d\n', int32(arr_sum));
    arr_mean = np.mean(arr);
    fprintf('numpy 数组均值: %.2f\n', double(arr_mean));
catch ME
    disp('numpy 未安装或调用失败, 跳过示例3。');
    disp(ME.message);
end

%% 示例4: 执行 Python 语句 (兼容写法)
if exist('pyrun', 'builtin')
    pyrun("msg = 'MATLAB 调用 Python 成功!'; print(msg)");
else
    py.builtins.print('MATLAB 调用 Python 成功!');
end
```

7.4 运行结果

```
>> run t7.m

ans =

PythonEnvironment - 属性:

    Version: "3.13"
    Executable: "C:\Users\16778\AppData\Local\Programs\Python\Python313\python.EXE"
    Library: "C:\Users\16778\AppData\Local\Programs\Python\Python313\python313.dll"
    Home: "C:\Users\16778\AppData\Local\Programs\Python\Python313"
    Status: Loaded
    ExecutionMode: InProcess
    ProcessID: "36904"
    ProcessName: "MATLAB"

py.math.sin(pi/6) = 0.500000
py.math.sqrt(2) = 1.414214
Python 字符串转大写: HELLO, MATLAB FROM PYTHON!
Python split 分词数: 4
numpy 数组求和: 15
numpy 数组均值: 3.00
MATLAB 调用 Python 成功!
```

7.5 结果分析

MATLAB 的 Python 接口通过 `py. 模块名. 函数名` 语法即可无缝调用 Python 生态中的丰富库，极大扩展了 MATLAB 的功能边界。需要注意数据类型的转换：Python 的 `float` 对应 MATLAB 的 `double`，需用 `double()` 显式转换；Python 字符串需用 `char()` 转回 MATLAB 字符串。对于大型数值计算，建议通过 `numpy` 传递数据，效率优于逐元素转换。

8 心得体会

本章作业围绕 MATLAB 的数据文件操作与外部接口展开。通过实践，深刻体会到高级 I/O 函数（`readtable`、`xlsread`）与低级 I/O 函数（`fopen`/`fread`）各有适用场景：前者适合规则数据，后者适合非结构化或二进制数据。COM 接口打通了 MATLAB 与 Office 软件的壁垒，而 Python 接口则让 MATLAB 能够借助 `numpy`、`pandas` 等生态完成更现代化的数据处理任务，二者均体现了 MATLAB 作为工程仿真平台的高度开放性与可扩展性。